Collection Framework

data → Object

↳ Query

arr = [3, 7, 10, 1, 4, 12, 24, 0]

Constant time → O(1)

Query : 24

↳ O(N)

6

arr (sort) [0, 1, 3, 4, 7, 10, 12, 24]

Query → O(1)

return arr[arr.length - 1];

= O(N)

Arrays

class LinkedList {

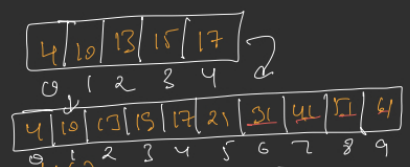
}

Dynamic Array

↳ int[] arr = new int[10];

arr[10] = Index Out of Bound.

```
Class DynamicArray {
    private int arr;
    DA( int size) {
        arr = new int[size];
    }
    add() {}        int el
    get( int index) {}
    insert( int idx,
            int el) {}
    delete() {}
}
```



add()

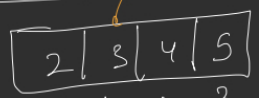
add(4);
add(10);
add(13);
add(15);
add(17);
add(21);

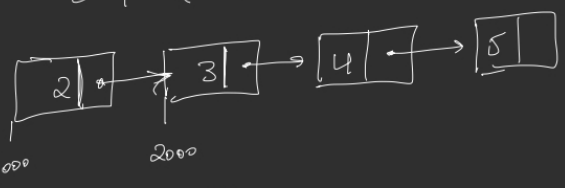
insert(1, 25);

ArrayList

Problem :

↳ LinkedList



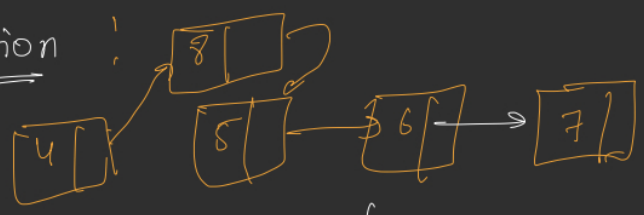



```
Class Node {
    int data;
    Node ref;
}
```

```
Class Node {
    int data;
    Node ref;
}
```

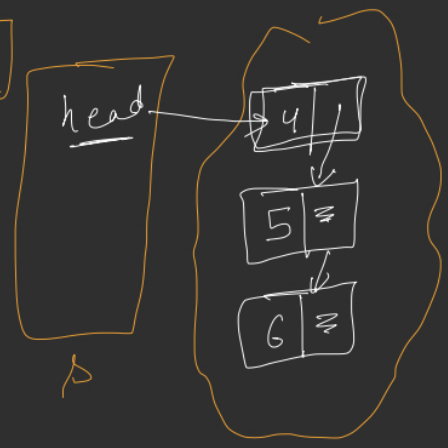
```
Node n1 = new Node();
n1.data = 4;

Node n2 = new Node();
n2.data = 5;
n1.ref = n2;
```

Insertion :




```
class LinkedList {
    private static class Node {
        int data;
        Node next;
    }
    Node head;
    public void add(int el){
    }
}
```

```
add(4);
add(5);
add(6);
add(7);
insert(8, 1);
```



LinkedList

Set :

↳ Duplicates are not allowed.

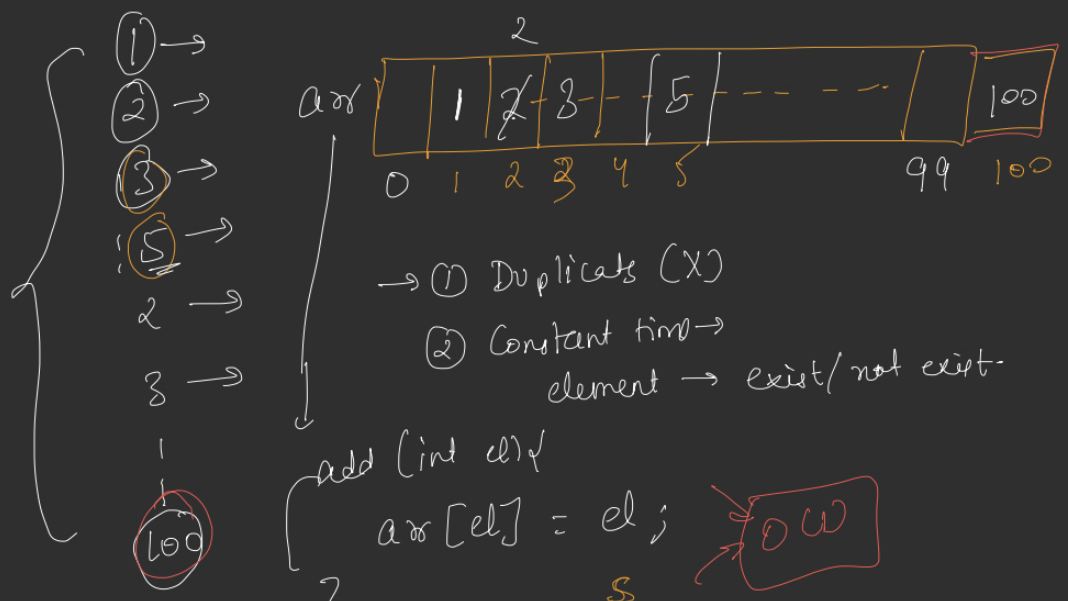
2

2 3 5 → 2, 3, 5

2 → 3 → 5 → 2 → 5

Constant time Query

2 | 3 | 5



→ (1) Duplicate (X)

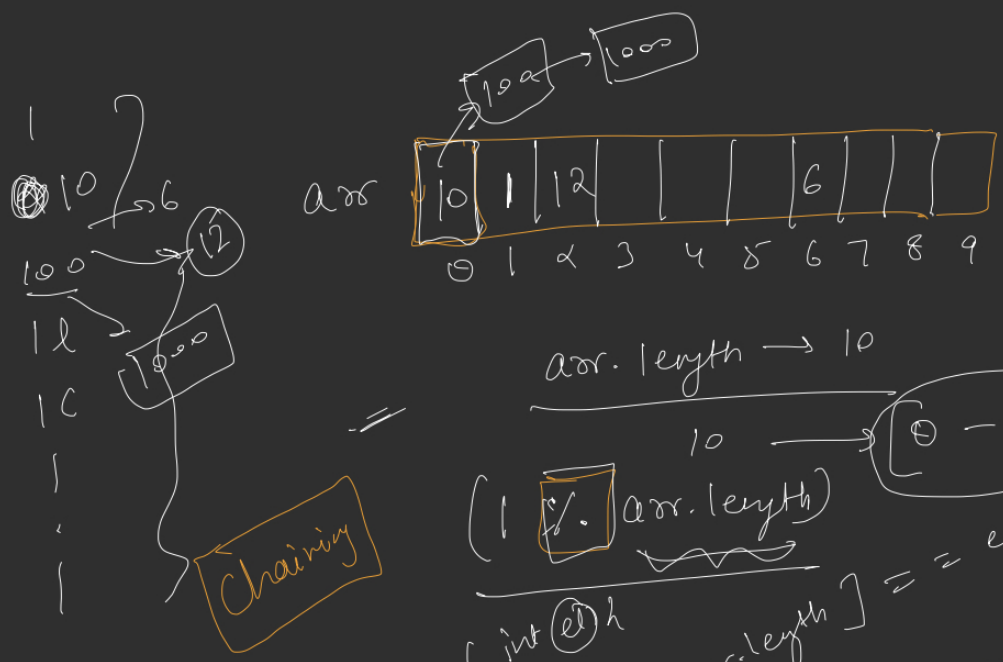
(2) Constant time →

element → exist/not exist.

```
add (int el){
    arr[el] = el;
}
```

O(1)

```
contains (int el){
    return arr[el] == el;
}
```



arr.length → 10

10 → [0 – 9]

(1 % arr.length)

```
contains(int el){
    arr[el % arr.length] == el);
}
```

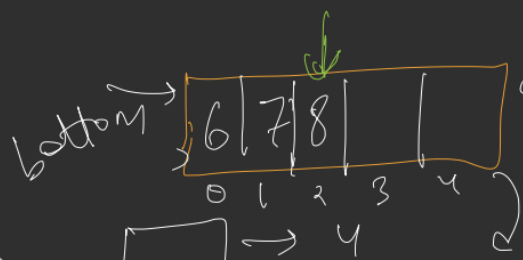
HashSet

LinkedHashSet

Hashes

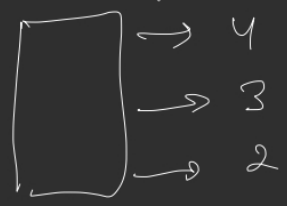
↳ hashCode

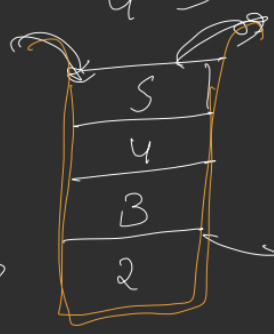
Stack / Queues :



2, 3, 4

2 →
3 →
4 →





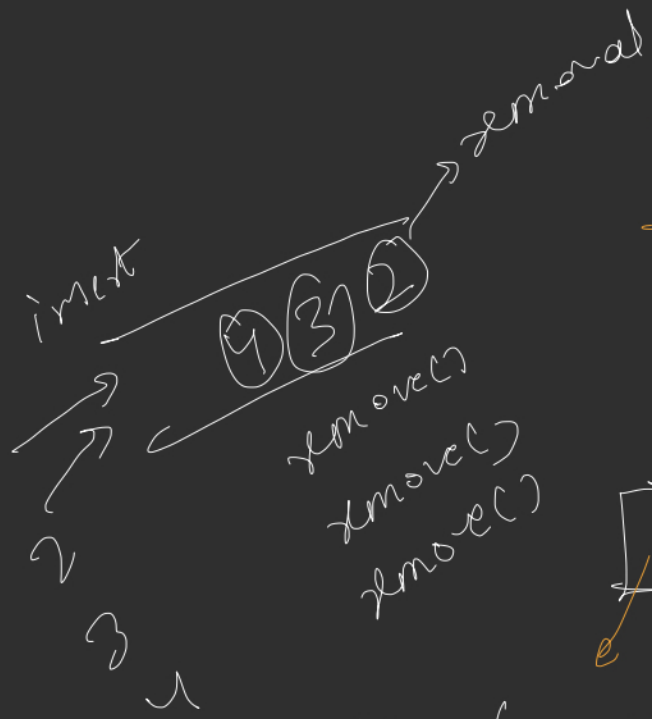
5, 4, 3, 2

LIFO

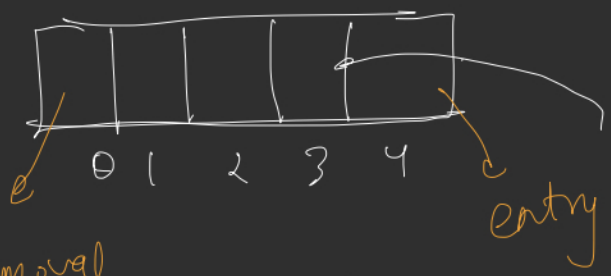
2
3
4

stack

```
class Stack {
    int arr;
    add(5);
    add(6)
    add(7)
    remove();
}
```



FIFO



Class Stack / Queue removal

int arr;

Queue

ArrayDeque

Stack

add(4)	remove();
add(5)	remove();
add(6)	remove();

Maps :

Key → value

Aditya → 101

Rohit → 102

Rohan → 103

O(1)

(1) Aditya → 101 O(1)

(2) Duplicate Keys(X)

Set → 1, 2, 3

↳ (K, V)

(Aditya, 101)

(Rohit, 102)

```
add (name, roll No) {
    Node n1 = new Node( , );
    (hashCode (name) % 5)
}
```

O(1)

```
Class Node {
    String Key;
    int value;
}
```

0	1	2	3	4
		n1		n2

```
get (Aditya) {
    101
}
```

Hash Map

Linked Hash Map

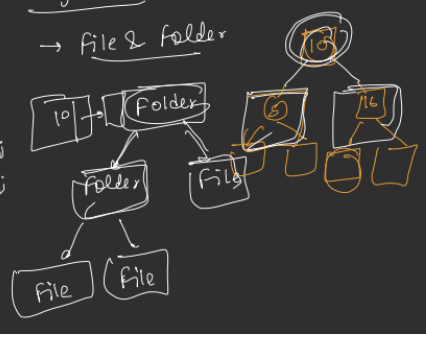
```
get (Key) {
    (hashCode (Key) % 5)
}
```

Tree :

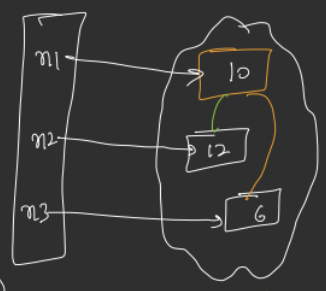
↳ BST : Binary Search Tree

→ File & Folder

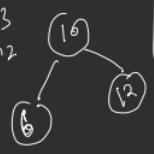
```
Class TreeNode {
    int data;
    TreeNode left;
    TreeNode right;
}
```



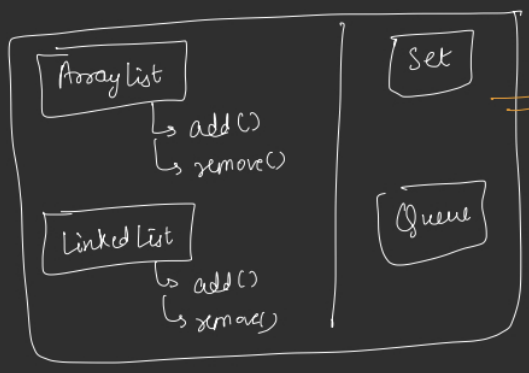
```
TreeNode n1 =
    new TreeNode();
        n1.data = 10;
```



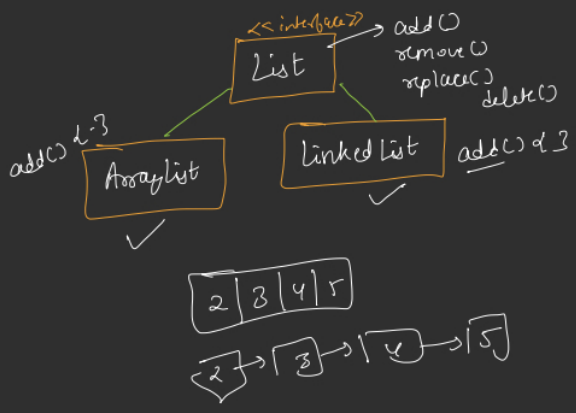
n1.left = n3
n1.right = n2

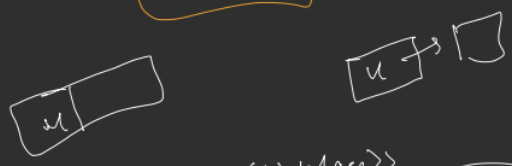
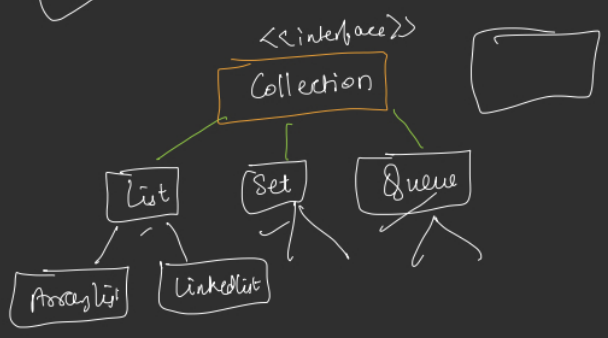


TreeSet
TreeMap → BST



→ Inheritance & Polymorphism




```
List<Integer> l = new ArrayList<>();
                  new LinkedList<>();
l.add(4);
```